

How to Use This Coding Guide

The technical round includes core Python - the classic 'Happy Number' was explicitly reported. These cover number theory and string handling. Every solution is executed and verified.

Problems

Problem 1: Happy Number

Difficulty: **Easy** | Pattern: **Cycle Detection (Set)** | Authenticity: **Verified Previous-Year**

Problem Statement

A happy number reaches 1 by repeatedly summing the squares of its digits; otherwise it loops. Decide if n is happy. (Explicitly reported in BI3 technical round.)

Input Format

A single integer n .

Output Format

True or False.

Constraints

$1 \leq n \leq 2^{31} - 1$

Sample Input

19

Sample Output

True

Python Solution

```
def is_happy(n):
    seen = set()
    while n != 1 and n not in seen:
        seen.add(n)
        n = sum(int(d) ** 2 for d in str(n))
    return n == 1

print(is_happy(19))
```

Step-by-Step Explanation

Repeatedly sum digit squares; a set detects loops. 19->82->68->100->1.

Dry Run

19->82->68->100->1 -> True

Time Complexity: **$O(\log n)/\text{step}$** | Space Complexity: **$O(\log n)$**

Problem 2: Factorial

Difficulty: *Easy* | Pattern: *Iteration* | Authenticity: *Community Reported*

Problem Statement

Compute $n!$ for a non-negative integer n .

Input Format

A single integer n .

Output Format

$n!$.

Constraints

$0 \leq n \leq 20$

Sample Input

5

Sample Output

120

Python Solution

```
def factorial(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

print(factorial(5))
```

Step-by-Step Explanation

Multiply 2..n; empty loop handles $0! = 1! = 1$. $5! = 120$.

Dry Run

$1 * 2 * 3 * 4 * 5 = 120$

Time Complexity: **$O(N)$** | Space Complexity: **$O(1)$**

Problem 3: Check Prime

Difficulty: *Easy* | Pattern: *Trial Division to $\sqrt{n}$* | Authenticity: *Community Reported*

Problem Statement

Decide whether n is prime.

Input Format

A single integer n .

Output Format

True or False.

Constraints

$1 \leq n \leq 10^9$

Sample Input

29

Sample Output

True

Python Solution

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True

print(is_prime(29))
```

Step-by-Step Explanation

No divisor up to $\sqrt{n}$ means prime; a larger factor pairs a smaller one.

Dry Run

29: no divisor up to 5 -> True

Time Complexity: **$O(\sqrt{N})$** | Space Complexity: **$O(1)$**

Problem 4: Palindrome Number

Difficulty: *Easy* | Pattern: *Digit Reversal* | Authenticity: *Community Reported*

Problem Statement

Decide whether an integer reads the same forwards and backwards (negatives are not).

Input Format

A single integer n .

Output Format

True or False.

Constraints

$-2^{31} \leq n \leq 2^{31} - 1$

Sample Input

121

Sample Output

True

Python Solution

```
def is_palindrome(n):
    if n < 0:
```

```

        return False
    original, rev = n, 0
    while n > 0:
        rev = rev * 10 + n % 10
        n //= 10
    return original == rev

print(is_palindrome(121))

```

Step-by-Step Explanation

Build the reverse via last-digit extraction and compare. $121 == 121$.

Dry Run

```
rev 1->12->121 == 121 -> True
```

Time Complexity: **$O(\log n)$** | Space Complexity: **$O(1)$**

Problem 5: GCD (Euclidean Algorithm)

Difficulty: **Easy** | Pattern: **Number Theory** | Authenticity: **Community Reported**

Problem Statement

Compute the greatest common divisor of two positive integers.

Input Format

Two integers a and b.

Output Format

The GCD.

Constraints

$1 \leq a, b \leq 10^9$

Sample Input

```
48 18
```

Sample Output

```
6
```

Python Solution

```

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

print(gcd(48, 18))

```

Step-by-Step Explanation

Replace (a,b) with (b, a mod b) until b is 0. $48,18 \rightarrow 18,12 \rightarrow 12,6 \rightarrow 6,0$.

Dry Run

```
->6
```

Time Complexity: **$O(\log \min)$** | Space Complexity: **$O(1)$**

Problem 6: Count Vowels

Difficulty: *Easy* | Pattern: *String Traversal* | Authenticity: *Community Reported*

Problem Statement

Count vowels (a,e,i,o,u, case-insensitive) in a string.

Input Format

A single line string.

Output Format

Number of vowels.

Constraints

$1 \leq \text{length} \leq 10^5$

Sample Input

Data Engineering

Sample Output

7

Python Solution

```
def count_vowels(text):  
    vowels = set('aeiou')  
    return sum(1 for ch in text.lower() if ch in vowels)  
  
print(count_vowels('Data Engineering'))
```

Step-by-Step Explanation

Lowercase and count vowel characters. 'Data Engineering' has a,a,e,i,e,e,i = 7.

Dry Run

Data:2 + Engineering:5 = 7

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 7: Reverse an Integer

Difficulty: *Easy* | Pattern: *Digit Manipulation* | Authenticity: *Research-Backed Company Style*

Problem Statement

Reverse the digits of a non-negative integer (drop leading zeros).

Input Format

A single integer n.

Output Format

The reversed integer.

Constraints

$0 \leq n \leq 10^9$

Sample Input

12300

Sample Output

321

Python Solution

```
def reverse_int(n):
    rev = 0
    while n > 0:
        rev = rev * 10 + n % 10
        n //= 10
    return rev

print(reverse_int(12300))
```

Step-by-Step Explanation

Pop last digits and build the reverse; trailing zeros vanish. 12300 -> 321.

Dry Run

rev 0->3->32->321

Time Complexity: **$O(\log n)$** | Space Complexity: **$O(1)$**

Problem 8: Sum of Digits

Difficulty: *Easy* | Pattern: *Digit Manipulation* | Authenticity: *Research-Backed Company Style*

Problem Statement

Compute the sum of the digits of a non-negative integer.

Input Format

A single integer n .

Output Format

The digit sum.

Constraints

$0 \leq n \leq 10^{12}$

Sample Input

12345

Sample Output

15

Python Solution

```
def digit_sum(n):
    total = 0
    while n > 0:
        total += n % 10
        n //= 10
    return total

print(digit_sum(12345))
```

Step-by-Step Explanation

Add each last digit while reducing the number. $1+2+3+4+5 = 15$.

Dry Run

$5+4+3+2+1=15$

Time Complexity: **$O(\log n)$** | Space Complexity: **$O(1)$**

BI3 Technologies

Python Fundamentals & Problems

Applied Role(s):

Data Engineer - Trainee

Online Assessment Preparation Material

Version 1.1 • Last Updated: July 2026

Every question carries an authenticity label. See the legend on the next page.

Table of Contents

How to Use This Coding Guide	1
Problems	1
Authenticity Legend & How to Read This Document	9

Authenticity Legend & How to Read This Document

Every question and problem in this document is labelled with an **authenticity classification** so you know how closely it maps to what candidates have actually reported. Labels are assigned conservatively based on research; authored practice questions are never presented as real exam questions.

Label	Meaning
Verified Previous-Year	The specific problem/pattern is documented from actual reported assessment experiences.
Frequently Repeated	The topic or pattern type is documented as recurring across multiple hiring cycles.
Community Reported	A widely shared, role-relevant pattern reported across placement communities and forums.
Research-Backed Company Style	Authored to match the researched difficulty, style and pattern - practice, not a real past question.

The correct option in every MCQ is highlighted in green and also restated below the question. Study the pattern, not just the answer.